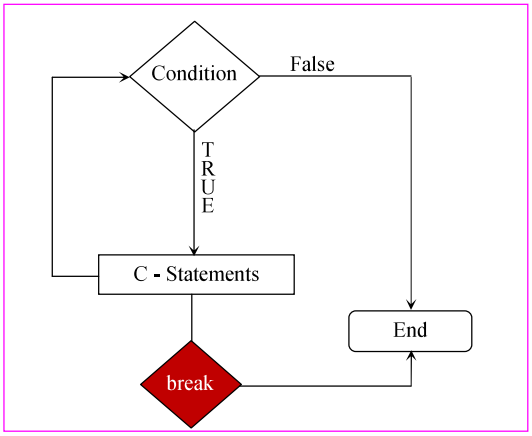
Break স্টেটমেন্ট-এর ব্যবহার

কোনো লুপের মধ্যে break স্টেটমেন্ট পেলে লুপ সেখানে তার কাজ শেষ করবে। গাড়ি চলা অবস্থায় ব্রেক করলে যেমন গাড়ি থেমে যায়, ঠিক তেমনি লুপের মধ্যে Break স্টেটমেন্ট পেলে লুপ থেমে যায়। যেমন :

প্রোগ্রাম-৫৬ : Break স্টেটমেন্টের ব্যবহার (লেখা প্রদর্শন)

```
#include<stdio.h>
int main(){
   int i;
    for(i=1; i<=10; i++)
     {
      printf("\ncomputer programming");
         if(i==6)
          {
          break;
          }
        }
    return 0;
}
```

প্রোগ্রামে i<=10 থাকার কারণে লেখাটি ১০ বার প্রদর্শিত হওয়ার কথা। কিন্তু if(i==6) থাকার কারণে computer programming লেখাটি ছয়বার প্রদর্শন করে break থাকার কারণে থেমে যাবে অর্থাৎ লেখাটি ছয়বার প্রদর্শিত হবে।



প্রোগ্রাম-৫৭ : Break স্টেটমেন্ট-এর ব্যবহার

```
#include <stdio.h>
int main()
{
int a = 10;
while( a < 20) {
printf("%d\n", a) ;
a++;
if( a > 15) {
break;
}
}
return 0;
}
```

প্রোগ্রামটি রান করালে 10 থেকে 15 সংখ্যা নতুন নতুন লাইনে নিচের মতো প্রদর্শিত হবে।

```
10  13  15
11  14
12
```

প্রোগ্রামে while(a<20) থাকায় 10 থেকে 19 পর্যন্ত সংখ্যা দেখানোর কথা। কিন্তু if(a>15) এর পরে break থাকায় প্রোগ্রামটি এ পর্যন্তই কাজ করে থেমে যাবে।

প্রোগ্রাম-৫৮ : Caseস্টেটমেন্টের সাথে break এর ব্যবহার

```
#include<stdio.h>
int main(){
int x;
printf( "Enter value of x:" );
scanf("%d",&x);
switch (x)      {
case 1:
printf("You have entered value 1 ");
break;
case 2:
printf("You have entered value 2 ");
break;
case 3:
printf("You have entered value 3 ");
break;
default:
printf("Input value is other than 1,2 & 3 ");
}
return 0;
}
```

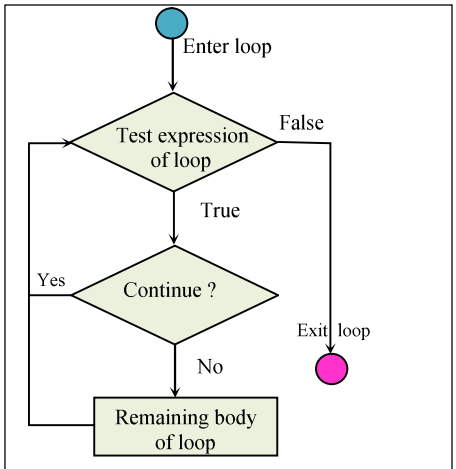
ইনপুটকৃত ভ্যালু যদি 1 হয়, তাহলে printf ফাংশনে You have entered value 1 লেখা প্রদর্শন করে break স্টেটমেন্টের কারণে প্রোগ্রাম থেমে যাবে। যদি 1 না পায়, তাহলে case 2: এ যাবে। সংখ্যাটি যদি 2 হয়, তাহলে এর কার্যক্রম করবে। সংখ্যাটি যদি 2 না হয় তাহলে পরবর্তী case 3: এ যাবে। যদি 3 হয় তাহলে printf ফাংশনে উল্লিখিত You have entered value 3 লেখা প্রদর্শন করে break হবে। আর যদি 3 না হয়, তাহলে default:G printf ফাংশনে উল্লিখিত Input value is other than 1, 2 & 3 লেখা প্রদর্শন করবে।

continue স্টেটমেন্ট এর ব্যবহার

লুপের ভেতরের কিছু স্টেটমেন্টকে মাঝে মাঝে এড়িয়ে যাওয়ার প্রয়োজন হয়। এক্ষেত্রে continue স্টেটমেন্ট ব্যবহৃত হয়। লুপের সাধারণ ফ্লো পরিবর্তন করার জন্য প্রোগ্রামিং এ continue স্টেটমেন্ট ব্যবহৃত হয়। continue স্টেটমেন্ট লুপের

মধ্যে কিছু স্টেটমেন্টকে এড়িয়ে যায়। সিদ্ধান্ত গ্রহণের জন্য ব্যবহৃত স্টেটমেন্ট যেমন- if...else স্টেটমেন্টে continue স্টেটমেন্ট ব্যবহৃত হয়। continue স্টেটমেন্ট if, else if, for, while ইত্যাদি ছাড়া সরাসরি কাজ করতে পারে না। এজন্য সাধারণত if, else... if স্টেটমেন্টের সাথে সম্পর্কিত শর্ত সাপেক্ষে কোনো লুপের পুনরাবৃত্তি করার জন্য continue স্টেটমেন্ট ব্যবহৃত হয়। সে ক্ষেত্রে continue এর সাথে if, else... if স্টেটমেন্টে যে শর্ত দেয়া হয় সে শর্ত এড়িয়ে যায়।

প্রোগ্রাম-৫৯ : continue স্টেটমেন্ট-এর ব্যবহার (উদাহরণ-১)



চিত্র :continue স্টেটমেন্টের ফ্লোচার্ট

```
#include <stdio.h>
 int main () {
   int a;
    for( a=10; a<=50; a++)
              {
               if( a==15 || a==25)
             continue ;
             printf("%d\t",a);
    }
   return 0;
}
```

প্রোগ্রামটি রান করালে 15 ও 25 ছাড়া 10 থেকে 50 পর্যন্ত সংখ্যা প্রদর্শিত হবে।
প্রোগ্রামে for(a=10; a<=50; a++) থাকায় 10 থেকে 50 পর্যন্ত সংখ্যা দেখানোর কথা। কিন্তু if(a==15 || a==25) continue থাকায় প্রোগ্রামটি continue-এর শর্ত a=15 এবং 25 ছাড়া continue করবে অর্থাৎ 15 ও 25 ছাড়া 10 থেকে 50 পর্যন্ত সংখ্যা দেখাবে।

আউটপুট :

10	11	12	13	14	16	17	18	19
21	22	23	24	26	27	28	29	30
32	33	34	35	36	37	38	39	40
42	43	44	45	46	47	48	49	50

প্রোগ্রাম-৬০ : continue স্টেটমেন্ট-এর ব্যবহার (উদাহরণ-২)

```
#include<stdio.h>
int main(){
int i;
for (i=1;i<=5;i++){
if(i==3)continue;
printf("\n HSC Exam");
}
return 0;
}
```

- প্রোগ্রামটি রান করালে 4 বার HSC Exam লেখাটি প্রদর্শিত হবে।
- প্রোগ্রামে for (i=1;i<=5;i++) থাকায় 5 বার HSC Exam লেখাটি প্রদর্শিত হওয়ার কথা। কিন্তু if(i==3) continue; থাকায় প্রোগ্রামটি continue এর শর্ত i==3 ছাড়া continue করবে অর্থাৎ HSC Exam লেখাটি 5 বার না দেখায়ে 4 বার দেখাবে।
- প্রোগ্রামটি continue করলে continue-এর শর্ত পূরণ হবে।

প্রোগ্রাম-৬১ : continue স্টেটমেন্ট-এর ব্যবহার। (উদাহরণ-৩)

```
//Example of continue control Statement*/
#include<stdio.h>
int main()
{
int x;
for( ; ; )  {
   printf("\nEnter a positive integer: ");
   scanf("%d", &x);
    if(x<=0)
     continue;
    else
     break;
   }
 printf("\n You have entered %d", x);
}
```

আউটপুট :
Enter a positive integer: -4
Enter a positive integer: 0
Enter a positive integer: 13
You have entered 13

- প্রোগ্রামটি রান করালে পজিটিভ (ধনাত্মক) সংখ্যা চাইব। প্রোগ্রামে দেয়া if(x<=0)এর শর্ত যদি সত্য নাও হয়, তবুও লুপ চলতে থাকবে এবং পজিটিভ (ধনাত্মক) সংখ্যা চাইতে থাকবে।
- যদি শর্ত পূরণ না করে তাহলে আর continue না করে নিচের দিকে গিয়ে else এ দেয়া break হবে এবং You have entere.... বার্তা দেখাবে। লুপের মধ্যে কোন শর্ত এবং টেস্টিং কন্ডিশন না থাকায় for লুপের মধ্যে (; ;) ব্যবহার করা হয়েছে।
- প্রথমে -4 এন্ট্রি করলে if(x<=0)এর শর্ত সত্য হবে। যেহেতু -4, 0 এর চেয়ে ছোট। তাই পজিটিভ নাম্বার দেয়ার কথা বলবে। ২য় বার 0 এন্ট্রি করলে এটিও শর্তানুসারে সত্য হবে এবং আবার পজিটিভ নাম্বার দেয়ার কথা বলবে। এবার 13 এন্ট্রি করলে শর্ত মিথ্যা হবে, তাই লুপের বাইরে গিয়ে You have entered 13 বার্তা দেখাবে।

অ্যারে (Array)

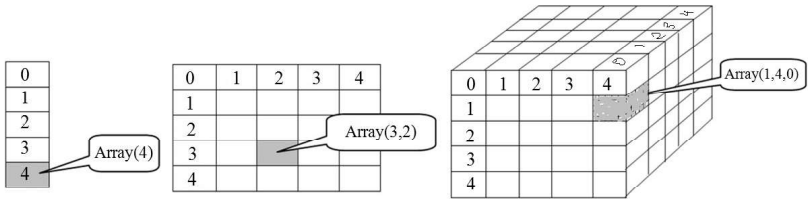
আমরা প্রোগ্রামে ভেরিয়েবল ডিক্লেয়ার করা জেনেছি। ধরা যাক, int num1 লেখা হলো। তাহলে num1 নামে একটি ইন্টেজার টাইপের ভেরিয়েবল (মেমোরি এড্রেস) সৃষ্টি হবে, যেখানে আমরা একটি পূর্ণ সংখ্যা ইনপুট করতে পারবো। যদি আমরা 50টি সংখ্যা ইনপুট করতে চাই, তাহলে num1, num2, num3... num50 এভাবে পঞ্চাশটি ভেরিয়েবল ডিক্লেয়ার করতে হবে, যা বেশ সময়সাপেক্ষ ও কষ্টের কাজ। তাই এটাকে সহজ করার জন্য নিচের মতো ডিক্লেয়ার করা যায় :

```
int num[50];
```
← এখানে 50 হলো অ্যারে সাইজ

পঞ্চাশটি ইন্টিজার টাইপের ভেরিয়েবল ডিক্লেয়ার করা যাবে এবং num[0], num[1], num[2]... num[49] এভাবে 50-টি মেমোরি এড্রেস সৃষ্টি হবে।

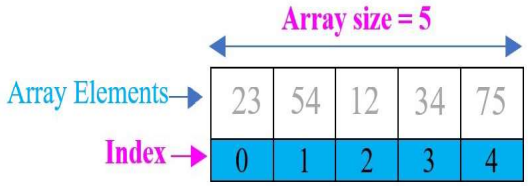
একই ধরনের ডেটা টাইপের গুচ্ছকে অ্যারে বলা হয়। অ্যারের উপাদানগুলো মেমোরিতে পাশাপাশি অবস্থান করে। অ্যারের নাম সংলগ্ন তৃতীয় বন্ধনীর '[ ]' মধ্যে অ্যারে সাইজ লেখা হয়, যা অ্যারে ভেরিয়েবলের সর্বোচ্চ ডেটার সংখ্যা নির্দেশ করে, এই সংখ্যাকে অ্যারের Index বলা হয় এবং অ্যারের প্রতিটি স্বতন্ত্র ভেরিয়েবলকে আলাদাভাবে অ্যারে উপাদান (Array element) বলা হয়। যেমন- int roll[4];

এখানে roll একটি int টাইপ অ্যারে। এই অ্যারেতে মোট চারটি রোল নাম্বার আছে, যা অ্যারের সাইজ নির্দেশ করে। এই অ্যারের ১ম ভেরিয়েবলটি হলো roll[0], ২য় ভেরিয়েবলটি হলো roll[1], ৩য় ভেরিয়েবলটি হলো roll[2] এবং ৪র্থ ভেরিয়েবলটি হলো roll[3]. অ্যারে একটি ডিরাইভড ডেটা টাইপ।



অ্যারে ব্যবহারের সুবিধা

১. একই ধরনের ডেটাগুলোকে একটি চলক দিয়ে প্রকাশ করা যায়।
২. অ্যারে প্রোগ্রামকে সহজ, সুন্দর ও ছোট করে।
৩. প্রোগ্রাম নির্বাহ দ্রুত হয়।
৪. অ্যারের উপাদানগুলো দ্রুত একসেস করা যায়।
৫. প্রোগ্রামের জটিলতা কমায়।



অ্যারে ব্যবহারের অসুবিধা

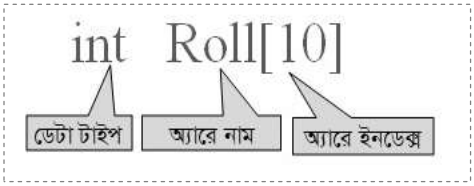
১. প্রোগ্রাম নির্বাহের সময় অ্যারের সাইজ পরিবর্তন করা যায় না।
২. প্রকৃত ডেটা অপেক্ষা অ্যারের সাইজ অনেক বেশি ঘোষণা করা হলে মেমোরির অপচয় হতে পারে।
৩. প্রকৃত ডেটা অপেক্ষা অ্যারের সাইজ কম ঘোষণা করা হলে অ্যারেতে ডেটার পর্যাপ্ত স্থান সংকুলান হয় না।
৪. বিভিন্ন টাইপের ডেটা অ্যারেতে রাখা যায় না।

অ্যারে ঘোষণার পদ্ধতি

সাধারণ ভেরিয়েবল ঘোষণার মতো ব্যবহারের পূর্বে ডেটা টাইপসহ অ্যারে ভেরিয়েবল ঘোষণার প্রয়োজন হয়। অ্যারে ঘোষণার সাধারণ ফরম্যাট হলো :

DataType ArrayName[ArraySize];

এখানে, DataType যে কোনো বৈধ ডেটা টাইপ, ArrayName প্রোগ্রামার কর্তৃক দেয়া অ্যারে ভেরিয়েবলের যেকোন বৈধ নাম এবং ArraySize পূর্ণসংখ্যায় প্রকাশিত কোন ধ্রুব মান, যাকে অ্যারে সাইজ



বা অ্যারে ইনডেক্স বলা হয়। অ্যারের নাম সংলগ্ন তৃতীয় বন্ধনীর '[ ]' মধ্যে পূর্ণসংখ্যা দ্বারা প্রকাশিত অ্যারে সাইজ বা অ্যারে ইনডেক্স অ্যারের সাইজ নির্ধারণ করে, যা অ্যারে ভেরিয়েবলে সংরক্ষিত সর্বোচ্চ ডেটার সংখ্যা নির্দেশ করে।

প্রোগ্রাম-৬২ : অ্যারে তৈরি করে এতে তিনটি সংখ্যা রেখে পর্দায় প্রদর্শন করা।

```
#include <stdio.h>
int main(){
int num[3];
num[0]=65;
num[1]=78;
num[2]=84;
printf("%d\n",num[0]);
printf("%d\n",num[1]);
printf("%d\n",num[2]);
return 0;
}
```

- এখানে num[3] নামের ইন্টিজার টাইপের অ্যারে তৈরি করা হয়েছে।
- অ্যারের প্রথম উপাদান থাকে ০-তম ঘরে, দ্বিতীয় উপাদান থাকে ১-তম ঘরে, তৃতীয় উপাদান থাকে ২-তম ঘরে, এরকমভাবে n-তম উপাদান থাকে (n-1) তম ঘরে। এই ঘরগুলোকে বলা হয় অ্যারের ইনডেক্স।
- সি প্রোগ্রামিং ভাষায় অ্যারের ইনডেক্স 0 থেকে শুরু হয়, 1 থেকে নয়। তাহলে num অ্যারেটিতে বিভিন্ন মান থাকবে নিচের মতো-

Value	65	78	84
Index	0	1	2

প্রোগ্রাম-৬৩ : অ্যারেতে বিভিন্ন মান অ্যাসাইন করা

```
#include <stdio.h>
int main(){
int num[] = {12,32,34,54,65,16,21};
int i, sum ;
sum=0;
for(i=0; i<7; i =i +1)
{
sum= sum+num[i];
}
printf("Sum of numbers = %d", sum);
return 0;
}
```

- এখানে এটি বলে দেয়া নেই যে কয়টি উপাদান থাকবে, তবে দ্বিতীয় বন্ধনীর ভেতরের উপাদানগুলোর সংখ্যা থেকেই কম্পাইলার বুঝে নেয় যে, অ্যারেতে কয়টি উপাদান থাকবে। তাই এ প্রোগ্রামে num[] টি হবে num[7]।
- প্রোগ্রামটি রান করালে অ্যারেতে অ্যাসাইন করা ৭টি নম্বরের যোগফল প্রদর্শিত হবে।
- for লুপের মধ্যে sum= sum+num[i] লেখাতে 7 এর নিচের অর্থাৎ num[] অর্থাৎ num[0]- num[6] এ রক্ষিত ৭টি নম্বরের যোগফল বের হবে।
- sum= sum+num[i] না লিখে sum += num[i] লেখা যায়।

প্রোগ্রাম-৬৪ : অ্যারে ব্যবহার করে সংখ্যা ইনপুট করে গড় নির্ণয় করা

```
#include <stdio.h>
int main()
{
    int marks[10], i, n, sum = 0, average;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    for(i=0; i<n; ++i)
    {
        printf("Enter number%d: ",i+1);
        scanf("%d", &marks[i]);
        sum= sum+marks[i];
    }
    average = sum/n;
    printf("Average = %d", average);
    return 0;
}
```

- প্রোগ্রামটি রান করালে Enter number of elements: লেখাটি আসবে। 5 লিখে এন্টার দিলে Enter number1: লেখাটি আসবে। ১ম নাম্বার 26 লিখে এন্টার দিলে Enter number2: লেখাটি আসবে। 34 লিখে এন্টার দিয়ে এভাবে বাকি ৩টি সংখ্যা ইনপুট করলে সংখ্যাগুলোর গড় প্রদর্শিত হবে।
- প্রোগ্রামে ইনপুট নেয়ার ফাংশন scanf এ marks[i] থাকায় এবং for লুপে i=0 লেখাতে marks[0] তে ১ম সংখ্যাটি রাখবে। i=i+1এর কারণে স্বয়ংক্রিয়ভাবে পরবর্তী অ্যারে ভেরিয়েবল marks[1] তৈরি হয়ে এতে ২য় সংখ্যা রাখবে।
- for লুপের মধ্যে sum= sum+marks[i] লেখাতে n এর মান 5 না হওয়া পর্যন্ত এন্ট্রিকৃত সংখ্যাগুলোর যোগফল হবে এবং একে 5 দিয়ে ভাগ করে Average বের হবে।

এতক্ষণ আমরা অ্যারেতে ইন্টেজার বা সংখ্যা ভ্যালু নিয়ে কাজ করেছি। এক্ষেত্রে একটি সংখ্যা রাখার জন্য একটি অ্যারে ব্যবহৃত হয়। একইভাবে ক্যারেক্টার টাইপের ভেরিয়েবলে একটি করে অক্ষর রাখা যায়, আবার স্পেস ছাড়া অনেক অক্ষরও রাখা যায়। একে স্ট্রিং বলা হয়। বিভিন্ন প্রোগ্রামিং ভাষায় স্ট্রিং-এর জন্য আলাদা ডেটা টাইপ থাকলেও সি-তে আলাদা কোন ডেটা টাইপ নেই। নিচের উদাহরণে সি-তে ক্যারেক্টার টাইপ ও স্ট্রিং-এর ব্যবহার দেখানো হলো :

প্রোগ্রাম-৬৫ : অ্যারেতে ক্যারেক্টার ও স্ট্রিং এর ব্যবহার

```
#include <stdio.h>
int main()
{
char str[70];
int i, count=0;
printf("Enter a string: \n");
scanf("%s",&str);
for(i=0; str[i]!='\0'; count++;++i)
printf("\nLength of string is: %d",count);
return 0;
}
```

প্রোগ্রামটি রান করালে স্ট্রিং (টেক্সট) চাইবে। স্ট্রিং টাইপ করে এন্টার করলে এর লেঙ্থ (শব্দ সংখ্যা) প্রদর্শিত করবে। একটি স্ট্রিং-এর শেষ অক্ষরটি হবে নাল ক্যারেক্টার (\0)। তাই কোন স্ট্রিং-এ যদি str[70] বলে দেয়া হয়, তাহলে ৬৯টি অক্ষর রাখা যাবে। শেষ ঘরটি নাল ক্যারেক্টারের জন্য বরাদ্দ রাখতে হবে।

str[i]!='\0' দিয়ে not at end of the string বুঝায়। \0 হলো Null ক্যারেক্টার, যা স্ট্রিং এর শেষ (end of the string) বুঝায়। যেমন- ক্যারেক্টার টাইপের অ্যারে char str[70]-এ 'Sample' রাখা হলে, এটি সংরক্ষিত হবে এভাবে- { 'S', 'a', 'm', 'p', 'l', 'e', **'\0'**} যেখানে \0 হলো Null ক্যারেক্টার এবং স্ট্রিং-এর শেষ। সুতরাং যখন str[i]!='\0', লেখা হবে তখন তা স্ট্রিং, এর শেষ পর্যন্ত না বুঝাবে। এখানে != হলো not equal to।

প্রোগ্রাম-৬৬ : অ্যারেতে সংরক্ষিত বিভিন্ন সংখ্যা হতে নির্দিষ্ট কোন সংখ্যা খুঁজে বের করা।

```
#include <stdio.h>
int main(){
int num[ ]={3,5,6,9,12,21,27,31};
int key, i, n;
n=8;
printf("Enter your number: \n");
scanf("%d",&key);
for(i=0;  i<n ;  i=i+1)
{
 if(num[i]= =key)
{
 printf("%d is found in the array.\n",key);
 break;
}
}
 if(i= =n)
{
  printf("%d is not found in the array.\n",key);
}
 return 0;
}
```

প্রোগ্রামটি রান করালে একটি সংখ্যা চাইবে। যে সংখ্যাটি টাইপ করে এন্টার করা হবে, সেটি যদি অ্যারেতে খুঁজে পায় তাহলে .. is found in the array দেখাবে এবং না ... is not found in the array. দেখাবে।

এন্ট্রি করা সংখ্যাটি খুঁজে পাওয়ার পর লুপ থেকে বের হয়ে যাওয়ার জন্য break স্টেটমেন্ট ব্যবহার করা হয়েছে। যদি অ্যারেতে key পাওয়া যায়, তাহলে আর খোঁজার প্রয়োজন নেই, তাই লুপ থেকে বের হয়ে যাবে।

যদি অ্যারে শেষ হওয়ার আগে key না পাওয়া যায়, তাহলে অ্যারের সব সংখ্যা পরীক্ষা করবে অর্থাৎ i এর মান n এর সমান হয়ে যাবে, তখন i<n শর্তটি মিথ্যা হয়ে যাওয়ায় প্রোগ্রামটি লুপ থেকে বের হয়ে যাবে।

তাই for লুপের ব্লকের বাইরে পরীক্ষা করবে i এর মান n-এর সমান কিনা। যদি সমান হয়, তাহলে বুঝতে হবে break স্টেটমেন্ট এক্সিকিউট হয়নি, অর্থাৎ সংখ্যাটি খুঁজে পাওয়া যায়নি। এভাবে ধারাবাহিকভাবে সার্চ করাকে লিনিয়ার সার্চ বলা হয়।

লিনিয়ার সার্চ ও বাইনারি সার্চ : একটি তালিকার মধ্যে একটি উপাদানকে সরলরৈখিকভাবে বা অনুক্রমিক খোঁজাকে লিনিয়ার সার্চ বলা হয়। এতে লাইন ধরে প্রথমটি দেখার পর দ্বিতীয়টি, দ্বিতীয়টি দেখার পর তৃতীয়টি এভাবে সার্চ করা হয়। সার্চিং অ্যালগরিদমের মধ্যে সবচেয়ে সহজ অ্যালগরিদম হলো লিনিয়ার সার্চ। সার্চিং এর আরেকটি পদ্ধতি হলো বাইনারি সার্চিং। এতে সুবিন্যস্ত তথ্য (ছোট থেকে বড় অথবা বড় থেকে ছোট এভাবে সাজানো)-এর ক্ষেত্রে এ পদ্ধতি ব্যবহৃত হয়।

অ্যারের ডাইমেনশন (Array Dimension)

একটি অ্যারের যেকোনো উপাদানকে শনাক্ত করার জন্য যত বা যতগুলো সংখ্যা প্রয়োজন হয়, তাকে ঐ অ্যারের ডাইমেনশন বা মাত্রা বলে। যেমন, int roll[4]; অ্যারের পাঁচটি উপাদানের যেকোন একটি শনাক্ত করার জন্য কেবল একটি সংখ্যা (0 থেকে 4 এর মধ্যবর্তী) প্রয়োজন হবে। C ল্যাঙ্গুয়েজে অ্যারের ইনডেক্স শুরু হয় 0 থেকে।

অ্যারের ডাইমেনশন-এর প্রকারভেদ

অ্যারের ডাইমেনশন এক বা একাধিক হতে পারে। ডাইমেনশনের ওপর ভিত্তি করে অ্যারে দু'ধরনের হতে পারে। যথা-

১. একমাত্রিক অথবা লিনিয়ার অ্যারে (1-D or One dimensional array or Linear array)
২. বহুমাত্রিক অথবা মেট্রিক্স অ্যারে (Multi dimensional or Matrix array)
   - দ্বিমাত্রিক অ্যারে (2-D or Two dimensional array)
   - ত্রিমাত্রিক অ্যারে (3-D or Three dimensional array)

একমাত্রিক অ্যারে (One Dimensional Array)

যদি কোনো অ্যারের ডেটাগুলো শুধুমাত্র একটি একক সারি বা কলাম আকারে উপস্থাপিত থাকে এবং একটি মাত্র সাবস্ক্রিপ্ট দ্বারা প্রকাশিত হয়, তবে সেই অ্যারেকে একমাত্রিক অ্যারে বলা হয়। সি ল্যাঙ্গুয়েজে একমাত্রিক অ্যারে তৈরির নিয়ম হলো-

```
data_type  variable_name [size]
```

যেমন, int roll[4]; এটি একটি একমাত্রিক অ্যারে। এখানে int হলো ডেটা টাইপ, roll হলো ভেরিয়েবলের নাম, এবং [4] হলো অ্যারে সাইজ। এই অ্যারের মধ্যে চারটি উপাদান রাখা যাবে।

দ্বিমাত্রিক অ্যারে (Two dimensional array)

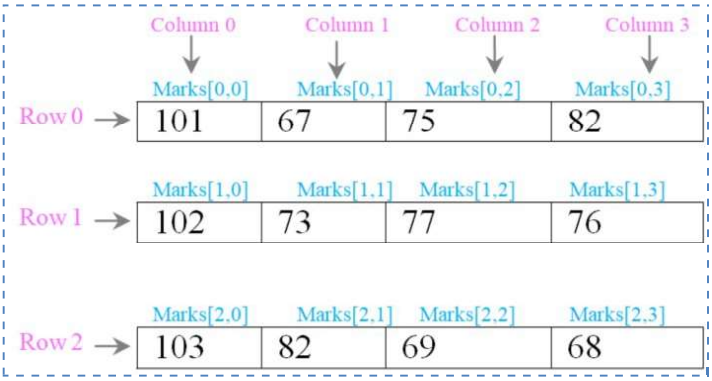
যদি কোনো অ্যারের ডেটাগুলো একক সারি ও কলাম আকারে উপস্থাপিত থাকে এবং দু'টি সাবস্ক্রিপ্ট দ্বারা প্রকাশিত হয় তবে সেই অ্যারেকে দ্বিমাত্রিক অ্যারে বলা হয়। যখন ডেটা টেবিলে রো ও কলাম আকারে মেমোরিতে সংরক্ষণ করার প্রয়োজন হয় তখন টু-ডাইমেনশনাল অ্যারে ব্যবহার করা হয়। যেমন-

Roll	Bang	Eng	ICT
101	67	75	82
102	73	77	76
103	82	69	68

এরূপ টেবিলের ন্যায় ডেটা সংরক্ষণ করার জন্য 3টি সারি ও 4টি কলাম তৈরি করতে হবে। সেজন্য marks নামের দ্বিমাত্রিক অ্যারে এর ঘোষণা হবে নিম্নরূপ :

```
int marks [3, 4];
```

এখানে marks এর সাইজের প্রথম সংখ্যাটি রো এবং ২য় সংখ্যাটি কলাম তৈরির জন্য ব্যবহার করা হয়েছে। উক্ত অ্যারেতে 3x4=12-টি লোকেশন আছে।



সি-তে দ্বিমাত্রিক অ্যারে ঘোষণার সাধারণ ফরম্যাট:

DataType ArrayName[RowSize][ColumnSize];

DataType - যে কোনো বৈধ ডেটা টাইপ।
ArrayName-প্রোগ্রামার কর্তৃক দেয়া অ্যারে ভেরিয়েবলের যেকোনো বৈধ নাম। RowSize ও ColumnSize পূর্ণসংখ্যায় প্রকাশিত কোনো ধ্রুব মান, যাদেরকে যথাক্রমে দ্বিমাত্রিক অ্যারের সারি সংখ্যা ও কলাম সংখ্যা বলা হয়।

উদাহরণ : int marks[3][4]; অ্যারেতে উপাদান কতটি?

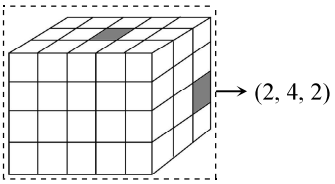
উত্তর : এই অ্যারের উপাদান সংখ্যা 12টি। যেমন :

	Column0	Column1	Column2	Column3
Row-0	marks (0, 0)	marks (0, 1)	marks (0, 2)	marks (0, 3)
Row-1	1, 0	1, 1	1, 2	1, 3
Row-2	2, 0	2, 1	2, 2	2, 3

∴ marks [3] [4] = 3(0 থেকে 2 = 3) × 4(0 থেকে 3 = 4) = 12

ত্রিমাত্রিক অ্যারে (Three dimensional array)

যদি কোনো অ্যারের ডেটাগুলো একক সারি, কলাম এবং প্রস্থ আকারে উপস্থাপিত থাকে ও তিনটি সাবস্ক্রিপ্ট দ্বারা প্রকাশিত হয়, তবে সেই অ্যারেকে ত্রিমাত্রিক অ্যারে বলা হয়। যেমন- int marks[2][3][2]; অথবা int marks[2,3,2]; int marks[2,3,2] এরকম ত্রিমাত্রিক অ্যারেতে 12-টি এলিমেন্ট রাখা যাবে। 0-1=2, 0-2=3, 0-1=2টি। সুতরাং [2,3,2] = 2×3×2=12.



সমস্যা : int marks[3, 4, 2] ত্রিমাত্রিক অ্যারেতে কয়টি এলিমেন্ট রাখা যাবে?
int marks[3, 4, 2] ত্রিমাত্রিক অ্যারেতে 24টি এলিমেন্ট রাখা যাবে।
0-2=3, 0-3=4, 0-1=2টি। সুতরাং [3, 4, 2] = 3×4×2=24

সমস্যা : বাম পাশের ছবির চিহ্নিত ঘরের অ্যাড্রেস কী?
ছবির চিহ্নিত ঘরের অ্যাড্রেস হলো [2, 4, 2]

অ্যারে ব্যবহার করে কতগুলো সংখ্যা হতে বড় সংখ্যাটি খুঁজে বের করার প্রোগ্রাম

অ্যারে ব্যবহার করে সহজেই কতগুলো সংখ্যার মধ্য থেকে বড় সংখ্যাটি নির্ণয় করা যায়। এজন্য অ্যারের প্রথম উপাদানটি s[0] বড় মনে করে পর্যায়ক্রমে অন্যান্য উপাদানসমূহের সাথে তুলনা করা হয়। পরবর্তী কোন উপাদান s[i] প্রথম উপাদান s[0] থেকে বড় হলে সেটিকে বড় ধরে পুনরায় এই প্রক্রিয়া চালানো হয়। এভাবে পর্যায়ক্রমিক কয়েকটি ধাপে কতগুলো সংখ্যা থেকে সবচেয়ে বড় সংখ্যাটি নির্ণয় করা যায়।

প্রোগ্রাম-৬৭ : কতগুলো সংখ্যা থেকে সবচেয়ে বড় সংখ্যা নির্ণয়

```
/*Finding maximum of a group of numbers */
#include<stdio.h>
int main()
{
int s[10],n,i,max;
printf("How many number ? ");
scanf("%d", &n);
printf("Enter %d integer numbers : \n", n);
for(i=0;i<n;i++)
scanf("%d", &s[i]);
max=s[0];

for(i=1;i<n;i++)
{
if(s[i] >max)
max=s[i];
else
max=max;
}
printf("%d is the maximum of given
numbers", max);
return 0;
}
```

অ্যারে ঘোষণা

Output:
How many number ? 5
Enter 5 integer numbers :
3 5 9 4 13
13 is the maximum of given numbers

প্রোগ্রামটি চালু করলে কয়টি সংখ্যা ইনপুট হিসেবে নিবে, সে সংখ্যা চাইবে। আমরা 5 লিখে এন্টার দিলে পরবর্তীতে পাঁচটি সংখ্যা ইনপুট দেয়ার বার্তা দেখাবে। 3, 5, 9, 4 ও 13 এ পাঁচটি সংখ্যা ইনপুট দিলে প্রোগ্রাম এই পাঁচটি সংখ্যা তার মধ্যে ডিক্লেয়ার করা s[10] অ্যারে এর s[0], s[1], s[2], s[3] ও s[4] এর মধ্যে রাখবে।

প্রথম ভেরিয়েবল s[0] এর মধ্যে রাখা 3 সংখ্যাটিকে ম্যাক্সিমাম ধরে s[i] অর্থাৎ s[1] এর মান (যেহেতু i এর প্রথম মান = 1) 5 এর 5 সাথে তুলনা করবে। s[1] অর্থাৎ এর মান 5। যেহেতু s[0] এর মান 3, s[i] এর মান 5 এর চেয়ে ছোট, তাই max=s[i]; অর্থাৎ এক্ষেত্রে ম্যাক্সিমাম সংখ্যা হলো 5।

এবার for(i=1;i<n;i++) এর জন্য i এর মান 1 বেড়ে যাবে। s[i] হবে s[2] । এতে রাখা 9 এর সাথে max=s[i] অর্থাৎ 5 এর তুলনা করবে। যদি দেখে if(s[i] >max) অর্থাৎ s[2] এর মান 9, max এ রাখা 5 এর চেয়ে বড়, তাহলে max=9 হবে।

এখন আবার for(i=1;i<n;i++)-এর জন্য i এর মান 1 বেড়ে যাবে। s[3] এ রাখা চতুর্থ সংখ্যার মান 4 এর সাথে max=s[i] অর্থাৎ 9 এর তুলনা করবে। যদি if(s[i] >max) অর্থাৎ s[i] অর্থাৎ s[3] এর মান 9, 4 এর চেয়ে বড় না হয়, তাহলে max= max-ই হবে। অর্থাৎ max=9-ই হবে।

এখন আবার for(i=1;i<n;i++) এর জন্য i এর মান 1 বেড়ে যাবে। s[4] এ রাখা ৫ম সংখ্যার মান 13 এর সাথে max অর্থাৎ 9 এর তুলনা করবে। if(s[i] >max) অর্থাৎ s[i] অর্থাৎ s[4] এর মান 13 যদি 9 এর চেয়ে বড় হয়, তাহলে max=s[i] অর্থাৎ 13 হবে।

চলক ও অ্যারের মধ্যে পার্থক্য

চলক (Variable)	অ্যারে (Array)
১. একটি চলকে শুধুমাত্র একটি ডেটা রাখা যায়।	১. একটি অ্যারেতে সমজাতীয় অনেক ডেটা রাখা যায়।
২. চলকের নামের সাথে এর সাইজ থাকে না।	২. অ্যারের নামের সাথে বন্ধনীর মধ্যে সাইজ লিখে দিতে হয়।
৩. চলক ঘোষণার নিয়ম হলো- DataType VariableName;	৩. অ্যারে ঘোষণার নিয়ম হলো- DataType ArrayName[ArraySize];
৪. উদাহরণ : int roll; এখানে roll নামে ইন্টিজার টাইপের ভেরিয়েবল ঘোষণা করা হয়েছে, যাতে 1টি পূর্ণ সংখ্যা রাখা যাবে।	৪. উদাহরণ : int roll[30]; এখানে roll নামে ইন্টিজার টাইপের অ্যারে ঘোষণা করা হয়েছে, যাতে 30টি পূর্ণ সংখ্যা রাখা যাবে।

ফাংশন (Function)

প্রোগ্রাম-৬৮ : বর্গমূল বের করা। (ফাংশনের ব্যবহার-১)

```
#include <stdio.h>
#include <math.h>
int main() {
float x, result;
printf("Enter number :");
scanf("%f",&x);
result = sqrt(x);
printf("The square root of %.2f is %.2f", x, result);
return 0;
}
```

প্রোগ্রামটি রান করালে Enter number : বার্তা দেখিয়ে একটি সংখ্যা চাইবে। আমরা যে সংখ্যাটি বর্গমূল (squre root) বের করতে চাই, সে সংখ্যাটি টাইপ করে এন্টার করলে সংখ্যাটির বর্গমূল দেখাবে। যেমন 81 টাইপ করলে বর্গমূল 9 দেখাবে।
$\sqrt{81}=9$

কিন্তু এখানে আমরা এভাবে না করে-
বর্গমূল = sqrt(সংখ্যা); ব্যবহার করেই মান পেতে পারি।
sqrt(); ফাংশনটি ব্যবহার করেই সংখ্যার বর্গমূল পেতে পারি। এই বর্গমূলের ফাংশনটি সি এর math.h হেডার ফাইলের লাইব্রেরিতে রয়েছে। তাই এটি হলো একটি লাইব্রেরি ফাংশন।

প্রোগ্রামে যখন কতগুলো স্টেটমেন্ট কোনো নামে একটি ব্লকের মধ্যে অবস্থান করে কোনো নির্দিষ্ট কাজ সম্পাদন করে তাকে ফাংশন বলা হয়। প্রতিটি সি প্রোগ্রাম হলো এক বা একাধিক ফাংশনের সমষ্টি। ফাংশন চেনার সহজ উপায় হলো ফাংশনের নামের শেষে এক জোড়া প্রথম বন্ধনী '()' থাকে, এই প্রথম বন্ধনীর মধ্যে ফাংশনটির প্যারামিটার (আর্গুমেন্ট) দেয়া হয়। প্রতিটি ফাংশনের একটি নাম থাকে, যে নামে কম্পাইলার তাকে শনাক্ত করে। প্রোগ্রাম নির্বাহের সময়ে কম্পাইলার যখন কোনো ফাংশন কল পায়, তখন মূল প্রোগ্রামের কাজ স্থগিত রেখে কল্ড ফাংশনে নির্বাহ শুরু করে এবং নির্বাহ শেষে মূল ফাংশনে প্রত্যাবর্তন করে পরবর্তী লাইন থেকে নির্বাহ চালিয়ে যায়। তবে এ প্রক্রিয়ায় অতিরিক্ত কিছুটা সময় ব্যয় হয়। ছোট প্রোগ্রামের জন্য সাধারণত ফাংশন ব্যবহার করা হয় না।

ফাংশনের প্রয়োজনীয়তা/সুবিধা

- ফাংশনের সাহায্যে প্রোগ্রামকে সংক্ষিপ্ত করা যায়।
- ফাংশন ব্যবহার করে একই ধরনের কাজের জন্য একই ধরনের স্টেটমেন্ট বার বার লেখার প্রয়োজন হয় না।
- প্রোগ্রামের ভুল নির্ণয় ও সংশোধন করা সহজ হয়।
- ব্যবহারকারী তার প্রয়োজন অনুযায়ী ফাংশন তৈরির কাজ করতে পারে।
- একই ফাংশন বিভিন্ন প্রোগ্রামে ব্যবহার করা যায়।
- প্রোগ্রাম দ্রুত নির্বাহ হয়।

ফাংশনের প্রকারভেদ :
সি ও সি++ এ ব্যবহৃত ফাংশনসমূহকে দু'টি ভাগে ভাগ করা হয়। যথা :
১. লাইব্রেরি ফাংশন বা বিল্ট ইন ফাংশন।
২. ইউজার ডিফাইন্ড ফাংশন বা ব্যবহারকারী বর্ণিত ফাংশন।

সি/সি++ প্রোগ্রামে ন্যূনতম main() নামে একটি ইউজার-ডিফাইন্ড ফাংশন থাকে। প্রোগ্রাম নির্বাহের শুরুতে main() ফাংশন স্বয়ংক্রিয়ভাবে নিয়ন্ত্রিত হয় এবং প্রয়োজনে এক বা একাধিক ফাংশন নিয়ন্ত্রণ করে। প্রোগ্রামে একটিই main() ফাংশন থাকতে পারে এবং main() ফাংশন ছাড়া অন্য যে কোনো ফাংশন যতবার প্রয়োজন কল করা যায়। সুতরাং সি/সি++ প্রোগ্রামে যত ফাংশনই থাকুক না কেন main() ফাংশনকে ঘিরেই যাবতীয় কার্যক্রম পরিচালিত হয়।

প্রোগ্রামিং ল্যাঙ্গুয়েজের গুরুত্বপূর্ণ একটা বিষয় হচ্ছে ফাংশন। ফাংশন দিয়ে খুব সহজে মডিউলার প্রোগ্রাম লেখা যায় অর্থাৎ সমগ্র প্রোগ্রামটাকে বিভিন্ন মডিউলে ভাগ করে নেয়া যায়। এতে করে প্রোগ্রাম অনেক বেশি গোছানো থাকে এবং ডিবাগিং ও পরবর্তী পর্যায়ের ডেভেলপমেন্ট অনেক সহজ হয়ে যায়।

লাইব্রেরি ফাংশন (Library Function)

যে ফাংশনের মান পূর্ব থেকে প্রোগ্রামে দেয়া থাকে, সে ফাংশনকে লাইব্রেরি ফাংশন বলা হয়। সি ভাষায় প্রয়োজনানুযায়ী লাইব্রেরি ফাংশন সরাসরি ব্যবহার করা যায়। লাইব্রেরি ফাংশনগুলোকে তাদের নিজস্ব ফরম্যাট অনুযায়ী main() ফাংশনে ব্যবহার করা যায়। printf(), scanf(), getch(), getchar(), abs(), sqrt(), sin(), cos(), tan(), rand() এ ধরনের লাইব্রেরি ফাংশনের উদাহরণ। লাইব্রেরি ফাংশনগুলোকে বিল্ট-ইন ফাংশনও বলা হয়। সি-তে বিভিন্ন গাণিতিক, যৌক্তিক ও অন্যান্য কার্যক্রম সম্পাদনের জন্য সি কম্পাইলারে এরূপ অনেক লাইব্রেরি ফাংশন আছে।

লাইব্রেরি ফাংশন ব্যবহারের নিয়ম

লাইব্রেরি ফাংশনগুলোর ঘোষণা তাদের হেডার (.h) ফাইলে এবং বিস্তারিত বর্ণনা সংশ্লিষ্ট লাইব্রেরি (.Lib) ফাইলে দেয়া থাকে। এজন্য সি প্রোগ্রামে কোন লাইব্রেরি ফাংশন ব্যবহার করলে প্রোগ্রামের শুরুতেই #include প্রিপ্রসেসিং ডিরেক্টিভ (preprocessing directive)-এর সাহায্যে সংশ্লিষ্ট হেডার ফাইল সংযুক্ত করতে হয়। হেডার ফাইল হলো মূলত .h এক্সটেনশন যুক্ত ফাইল, যেখানে বিভিন্ন লাইব্রেরি ফাংশনের প্রটোটাইপ উল্লেখ থাকে। লাইব্রেরি ফাংশনগুলোর প্রটোটাইপ তার হেডার ফাইলে বর্ণিত থাকে। প্রটোটাইপ হলো ফাংশনের বডি লেখা বা তাকে ব্যবহার করার আগে কী নামে একটি ইউজার ডিফাইন্ড ফাংশন তৈরি করা হবে, যাকে কল করার জন্য কী কী আর্গুমেন্ট দিতে হবে এবং যা কী টাইপের ভ্যালু রিটার্ন দিবে তা আগে কম্পাইলারকে জানিয়ে রাখা। প্রোগ্রামে কোনো হেডার ফাইল সংযুক্ত করা হলে কম্পাইলার প্রোগ্রাম কম্পাইল করার সময় সংযুক্ত ফাইলের উপাদানগুলো সংশ্লিষ্ট লাইব্রেরি ফাইল থেকে কপি করে। এরূপ সংযোগের ফলে কম্পাইলারের এমন মনে হয় যে, সংযুক্ত ফাইল যেন সংযোগকারী ফাইলেরই অংশবিশেষ।

উল্লেখ্য, প্রোগ্রামের শুরুতে #include স্টেটমেন্টের মাধ্যমে কোনো হেডার ফাইল সংযুক্ত করা হলে তা ঐ হেডার ফাইলে বর্ণিত সকল ফাংশনের জন্য কাজ করে।

প্রোগ্রাম-৬৯ : স্ট্রিং-এর দৈর্ঘ্য (ক্যারেক্টার সংখ্যা) জানা

```
#include <stdio.h>
#include <string.h>
int main()
{
char txt[70];
int length;
printf("Enter string :");
scanf("%s", &txt);
length=strlen(txt);
printf("%s has %d characters.\n",txt,length);
return 0;
}
```

প্রোগ্রামটি রান করালে স্ট্রিং (টেক্সট) চাইবে। স্ট্রিং টাইপ করে এন্টার করলে এর লেঙ্থ (শব্দ সংখ্যা) প্রদর্শিত হবে। strlen() ফাংশন স্ট্রিং-এর লেঙ্থ বের করে সংখ্যায় প্রকাশ করবে। যেমন- Enter string : এ টাইপ করলে Bangla টাইপ করলে আউটপুট হিসেবে দেখাবে। কারণ Bangla এ স্ট্রিং এ পাঁচটি ক্যারেক্টার (অক্ষর) রয়েছে।

অ্যারে ব্যবহার করে আমরা কোন স্ট্রিং-এর লেঙ্থ বা দৈর্ঘ্য বের করার প্রোগ্রাম তৈরি করেছি। এখন ফাংশন ব্যবহার করে স্ট্রিং এর লেঙ্থ বের করা দেখব। strlen() ফাংশনটি ব্যবহার করে স্ট্রিং-এর লেঙ্থ বের করা যায়। strlen() এর হেডার ফাইল হলো string.h।

প্রোগ্রাম-৭০ : ২-টি স্ট্রিং এর মধ্যে তুলনা করা

```
#include <stdio.h>
#include <string.h>
int main(){
char s1[70], s2[70];
int value;
printf("Enter string-1 :");
scanf("%s", &s1);
printf("Enter string-2 :");
scanf("%s", &s2);
value = strcmp(s1, s2);
if (value= =0)
{
printf("%s and %s are equal.\n",s1,s2);
}
else if (value > 0)
{
printf("%s is greater than %s.\n", s1, s2);
}
else
{
printf("%s is smaller than %s.\n", s1, s2);
}
return 0;
}
```

এখানে আমরা আরেকটি strcmp() লাইব্রেরি ফাংশনটি ব্যবহার করে দুইটি স্ট্রিং এর মধ্যে তুলনা করা দেখবো। strcmp() এর হেডার ফাইল হলো string.h।

প্রোগ্রামটি রান করালে Enter string-1 প্রদর্শিত হবে। প্রথম স্ট্রিং (টেক্সট) টাইপ করে এন্টার করলে Enter string-2 প্রদর্শিত হবে। দ্বিতীয় স্ট্রিং (টেক্সট) টাইপ করে এন্টার করলে স্ট্রিং-১ এবং স্ট্রিং-২ দুটি সমান, নাকি বড়-ছোট সেটি বের করবে।

এক্ষেত্রে strcmp() এর কাজ হলো স্ট্রিং দুটি তুলনা করে সমান হলে 0 রিটার্ন করা, প্রথমটি বড় হলে 1 রিটার্ন করা আর প্রথমটি ছোট হলে -1 রিটার্ন করা।

এখানে বড়-ছোট মানে দৈর্ঘ্যে বড়-ছোট নয় বরং লেক্সিকোগ্রাফিক্যালি (lexicographically) বড়-ছোট কিনা তা বোঝানো হয়েছে। এর মানে হচ্ছে আভিধানিক (ডিকশনারি) ক্রমে সাজালে যে স্ট্রিংটি আগে আসবে তাকে ছোট আর যেটি পরে আসবে তাকে বড় বলে বিবেচনা করা হবে।

ইউজার ডিফাইন্ড ফাংশন (User Defined Function)

সি কম্পাইলারে অনেক বিল্ট-ইন/লাইব্রেরি ফাংশন থাকা সত্ত্বেও প্রোগ্রাম রচনার সময় চাহিদা অনুযায়ী সব রকম ফাংশন পাওয়া যায় না। সেক্ষেত্রে প্রোগ্রামার তার নিজস্ব প্রয়োজন এবং প্রজ্ঞা অনুযায়ী যে সকল ফাংশন তৈরি করে প্রোগ্রামে ব্যবহার করেন সেগুলোকে ইউজার ডিফাইন্ড ফাংশন বলা হয়।

ফাংশনের উপাদান :

সি প্রোগ্রামে ইউজার ডিফাইন্ড ফাংশন ব্যবহার করতে গেলে সাধারণত চারটি বিষয় বিবেচনা করতে হয়। যথা-

(১) ফাংশন বর্ণনা
(২) ফাংশন কল
(৩) ফাংশনের প্রোটোটাইপ
(৪) ফাংশনের রিটার্ন টাইপ ও রিটার্ন স্টেটমেন্ট

ইউজার ডিফাইন ফাংশন ঘোষণার ফরমেট :

```
return_type func_name (parameters)
{
function_body
return value
}
```

ফাংশন কল (Function Call)

যখন একটি ফাংশন অপর কোনো ফাংশনকে ব্যবহার করে, তখন তাকে ব্যবহারকারী বা মূল ফাংশন এবং যে ফাংশন ব্যবহার করা হয় তাকে ব্যবহৃত বা কল্ড ফাংশন বলা হয়। আর এই প্রক্রিয়ার নাম ফাংশন কল। ফাংশন কল প্রক্রিয়ায় একটি ফাংশনের সাথে অপর এক বা একাধিক ফাংশন সংযুক্ত করা হয়। ফাংশন কল একটি স্টেটমেন্ট। সুতরাং এর শেষে অবশ্যই সেমিকোলন (;) থাকতে হবে। এক জোড়া প্রথম বন্ধনী '()'দেখে যেমন ফাংশন চেনা যায়; তেমনি এর জোড়া প্রথম বন্ধনীর শেষে অতিরিক্ত একটি সেমিকোলন দেখে ফাংশন কল বুঝা যায়। যে কোনো সি/সি++ প্রোগ্রামের main() ফাংশন অপর এক বা একাধিক লাইব্রেরি কিংবা ইউজার-ডিফাইন্ড ফাংশন কল করতে পারে। এরূপ ইউজার-ডিফাইন্ড ফাংশন আবার এক বা একাধিক লাইব্রেরি কিংবা ইউজার-ডিফাইন্ড ফাংশন কল করতে পারে। এভাবে এক ফাংশন এক বা একাধিকবার অন্য যে কোনো ফাংশন ব্যবহার করতে পারে।

ইউজার ফাংশন তৈরি করা

সি/সি++ প্রোগ্রামে সাধারণত কোনো ইউজার-ডিফাইন্ড ফাংশন কল বা ব্যবহার করার জন্য ব্যবহারকারী main() ফাংশনের উপরে তা বর্ণনা করে থাকেন। কিন্তু অনেক সময় বড় কোনো ফাংশনের ক্ষেত্রে বা একাধিক ফাংশনের ক্ষেত্রে এরূপ বর্ণনা করা সম্ভব হয় না। কারণ এতে main() ফাংশনের বর্ণনা অনেক পরে থাকায় তাতে ভুলের সম্ভাবনা থাকে। আবার ব্যবহারকারী কর্তৃক ফাংশনের পূর্বে বর্ণিত এরূপ ফাংশনকে গ্লোবাল ফাংশন হিসেবে বিবেচিত হয়, যাতে করে ফাংশনের প্রাইভেসি হ্রাস পায়। এজন্য সি/সি++ প্রোগ্রামে কোনো ইউজার-ডিফাইন্ড ফাংশন ব্যবহার করার পূর্বে ব্যবহারকারী বা main() ফাংশনের উপরে তা ঘোষণা করা হয় এবং ব্যবহারকারী বা main() ফাংশনের নিচে সুবিধাজনক স্থানে বর্ণনা করা হয়। ফাংশনের এরূপ ঘোষণাকে ফাংশন প্রোটোটাইপ বলা হয়। নিচের প্রোগ্রামটি লক্ষ করা যাক।

প্রোগ্রাম-৭১ : ইউজার ফাংশনের ব্যবহার-১

```
#include<stdio.h>
int hello()
{
   printf("Welcome\n");
   printf("C Programming\n");
   printf("World\n\n\n");
}
```

প্রোগ্রামটি রান করলে ফলাফল আসবে না। কারণ সি কম্পাইলার main(){ ফাংশন না পেলে কাজ শুরু করে না। hello() হলো আমাদের তৈরি করা একটি ফাংশন, যার মান হলো তিনটি printf() এর স্টেটমেন্ট। hello() ফাংশনটি কল করলে এর মান পাবো-

```
Welcome
C Programming
World
```

প্রোগ্রাম-৭২ : ইউজার ডিফাইন ফাংশনের ব্যবহার-২

আমরা যদি প্রোগ্রামটি এভাবে লেখি :

```
#include<stdio.h>
int hello()
{
   printf("Welcome\n");
   printf("C Programming\n");
   printf("World\n\n\n");
}
main() int
 {
  hello();
  hello();
}
```

উক্ত প্রোগ্রামটি রান করালে প্রথমে ডিক্লেয়ার করা ফাংশনটিতে main() ফাংশন নেই বিধায় কাজ করবে না। অর্থাৎ এর ভেতর লেখা তিনটি স্টেটমেন্ট (প্রিন্ট নির্দেশ) কাজ করবে না। কিন্তু main() এর অধীনে hello() ফাংশন দুইটি কল করার কারণে এগুলোর স্টেটমেন্ট কাজ করবে এবং ফলাফল হিসেবে দেখাবে:

```
Welcome
C Programming
World

Welcome
C Programming
World
```

কারণ এখানে দুটি ফাংশন ব্যবহার করা হয়েছে। এই hello() হলো ব্যবহারকারী কর্তৃক তৈরিকৃত বা ইউজার ডিফাইন ফাংশন। এভাবে ইউজার কর্তৃক প্রোগ্রামে কোনো ফাংশন তৈরি করে পরবর্তীতে শুধু ফাংশনটি কল করে অনেক কাজ সহজে করা যায়।

প্রোগ্রাম-৭৩ : ইউজার ডিফাইন ফাংশনের ব্যবহার-৩

```
#include <stdio.h>
int addition(int n1, int n2, int n3)
{
int sum;
sum = n1+n2+n3;
return sum;
}
int main(){
int a,b,c, add;
printf("Enter three number : ");
scanf("%d %d %d",&a,&b,&c);
add = addition(a,b,c);
printf (" Summation of three number : %d",
add);
return 0;
}
```

এখানে রিটার্ন টাইপ int হওয়াতে এটি পূর্ণসংখ্যা বা ইন্টিজার টাইপের ডেটা রিটার্ন করবে।

addition হলো ফাংশনটির নাম। এটি sum অর্থাৎ প্যারামিটারে উল্লিখিত তিনটি সংখ্যার যোগফল রিটার্ন করে।

প্রোগ্রামের মেইন ফাংশনের মধ্যে addition নামের ফাংশনটি কল করা হয়েছে, যা তিনটি সংখ্যার যোগফল নির্ণয় করে। এ ফাংশনের আর্গুমেন্ট হিসেবে শুধুমাত্র সংখ্যা তিনটির ভ্যালু দিলেই এদের যোগফল রিটার্ন করবে।

এভাবে প্রোগ্রামের ভিতরে আমরা বার বার যোগফল=১ম সংখ্যা+২য় সংখ্যা+৩য় সংখ্যা না লিখে addition ফাংশনকে কল করে তিনটি সংখ্যার যোগফল সহজেই বের করতে পারি।

রিকার্সিভ ফাংশন (Recursive Function)

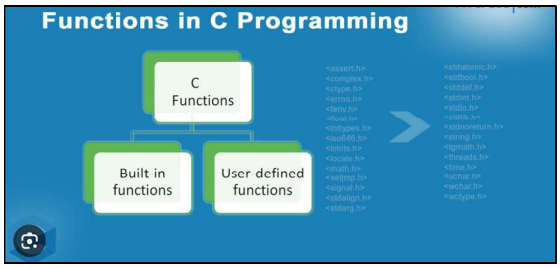
সি প্রোগ্রামে কোনো ফাংশন যখন স্বয়ংক্রিয়ভাবে নিজেকে নিজে call করে, তখন সেই ফাংশনকে রিকার্সিভ ফাংশন এবং এই প্রক্রিয়াকে রিকার্সিভ বলা হয়। গণিতশাস্ত্রের সিরিজকে রিকার্সিভ ফাংশনের মাধ্যমে সহজে সমাধান করা যায়। প্রতিটি রিকার্সিভ ফাংশনের একটি টার্মিনেটিং কন্ডিশন থাকতে হয়। কোনো একটা প্রবলেমকে যদি ভেঙে ছোট ছোট প্রবলেমে ভাগ করা যায়, আর ছোট ছোট প্রবলেমের সলিউশনের ওপর ভিত্তি করে মূল সলিউশন বের করা যায়, তাহলে সেটাকে রিকার্শন দিয়ে সলভ করা বলা যায়।

রিকার্সিভ ফাংশন ব্যবহারের সুবিধা :

- কম সংখ্যক ভেরিয়েবল লাগে বিধায় প্রোগ্রামের জটিলতা কমে।
- প্রোগ্রাম সহজ ও সুন্দর হয়।
- অপ্রয়োজনীয় ফাংশন কলিং কমায়।

রিকার্সিভ ফাংশন ব্যবহারের অসুবিধা :

- প্রোগ্রামে লুপের সংখ্যা বেড়ে যায়।
- প্রোগ্রাম নির্বাহে সময় বেশি লাগে।
- স্ট্যাক স্পেসের পরিমাণ বেশি লাগে।





addition ফাংশনটি শুধুমাত্র এই প্রোগ্রামে যতবার ইচ্ছে ততবার ব্যবহার করা যায়। কিন্তু এটিকে যদি আমরা অন্যান্য প্রোগ্রামে ব্যবহার করতে চাই, তাহলে এর জন্য একটি হেডার ফাইল তৈরি করতে হবে। এক্ষেত্রে অন্য যে কোনো প্রোগ্রামে #include এর সাথে হেডার ফাইলটি ইনক্লুড করলেই addition ফাংশনটি কল হবে। অর্থাৎ addition ফাংশনটির বর্ণনা কোড (ফাংশন বডি) টাইপ করতে হবে না।

ধরা যাক, addition ইউজার ডিফাইন ফাংশনটির হেডার ফাইলের নাম দেয়া হলো addi.h। এটিকে নতুন কোন প্রোগ্রামে ব্যবহার করার জন্য মেইন ফাংশনের উপরে #include "addi.h" লিখে দিতে হবে। এখানে < > এর পরিবর্তে ডাবল কোটেশন "" দেয়াতে এটি ইউজার ডিফাইন বুঝাবে।

হেডার ফাইল : সি প্রোগ্রামের সোর্স কোডের লিঙ্ক সেকশনে লাইব্রেরিতে সংরক্ষিত যে সব ফাইলকে যুক্ত করা হয় তাদেরকে header ফাইল বলা হয়। এই ফাইলের এক্সটেনশন হলো .h। যেমন- stdio.h সি প্রোগ্রামিং ভাষার একটি হেডার ফাইল। উক্ত ফাইলে printf() বা scanf() ফাংশনের বর্ণনা রয়েছে। ফলে printf() ফাংশন প্রোগ্রামে ব্যবহার করলে stdio.h হেডার ফাইলটি প্রোগ্রামে অন্তর্ভুক্ত করতে হয়। এখানে #include<stdio.h> লিখে প্রোগ্রামে উক্ত হেডার ফাইলটি সংযুক্ত করা হয়েছে।

সি প্রোগ্রামিং ভাষায় লিখিত প্রোগ্রামে #include<stdio.h> আবশ্যক কেন?

একটি আদর্শ প্রোগ্রামে ইনপুট, প্রসেস ও আউটপুট এর ব্যবস্থা থাকতে হয়। সি প্রোগ্রামিং ভাষায় লেখা একটি প্রোগ্রামে ইনপুট এবং আউটপুট এর জন্য যথাক্রমে scanf() এবং printf() নামক লাইব্রেরী ফাংশন ব্যবহার করা হয়। এই ফাংশন দুটির জন্য #include<stdio.h> হেডার ফাইল ব্যবহার করা হয়। তাই বলা যায়, সি প্রোগ্রামিং ভাষায় লিখিত প্রোগ্রামে #include<stdio.h> আবশ্যক।